

Claude Code Quick Start and Anthropic Product Offering for Academic Environments

Installation, authentication, and product-tier comparison of Claude Code and Anthropic API access options for academic statistics departments operating multi-agent project templates on Ubuntu Linux

datalab@unimib.it

Project:	dve-sample-r
Document:	Python-Q7-Claude-Configuration.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-r/notes/howtos/
Date:	April 21, 2026

Keywords: GEN, Claude-Code, Anthropic-API, multi-agent, nvm, Ubuntu, academic-deployment

Abstract

This document addresses two related objectives concerning the deployment of Anthropic tooling in an academic statistics department running a shared multi-agent project template on Ubuntu 24.04.

The first part presents a practical quick-start guide for `claude-code`, Anthropic's CLI coding assistant, installed via a user-level `nvm`-managed `npm` environment. It covers installation, authentication and model configuration for zero-cost trial use, installation verification, and a worked interaction example assuming `uv`-managed Python tooling (`uv run pytest`, `uv run ruff`). The role of `.claude/CLAUDE.md` as a project-context entry point—importing shared agent instructions from `.agents/AGENTS.md` via an `@AGENTS.md` directive—is explained, together with how the `.agents/skills/` directory is surfaced to Claude Code, enabling skill reuse across Claude Code, Gemini CLI, Cursor, and VS Code Copilot without duplication.

The second part provides a structured comparison of Anthropic access tiers—Claude.ai Free, Pro, Team, and Enterprise, the pay-as-you-go Anthropic API, and Claude Code as a product layer—evaluated against criteria relevant to an academic organisation: monthly cost, free-tier availability, model access (Haiku / Sonnet / Opus), context window, rate limits, API key requirements, team and organisation support, and data-privacy policy.

A recommended proof-of-concept path for a department seeking zero-cost initial evaluation prior to any paid commitment is derived from this comparison. Together, the two sections constitute a self-contained onboarding reference for researchers and students adopting Claude Code within a pre-existing multi-agent template infrastructure.

Contents

TOC	5
Q:1	6
Q:1 - Claude Code - Quick Start Guide	6
Role	6
Context	6
Primary Objective	6
Secondary Objective	7
Output Format	7
A:1 (Claude)	8
Prerequisites	8
Step 1 — Install the Claude Code CLI	8
Step 2 — Authentication and Zero-Cost Access Options	9
Important: Free Tier Does Not Include Claude Code	9
Option A — Native Anthropic Authentication (Pro/Console)	9
Option B — Free Evaluation via Google AI Studio (Zero Cost)	9
Step 3 — Verify Installation and Project Context	10
Step 4 — Worked Interaction Example	10
4.1 — Start Claude Code in the project root	10
4.2 — Explain a failing test	10
4.3 — Fix and lint in one pass	11
Step 5 — How CLAUDE.md and AGENTS.md Work Together	11
Secondary Objective — Skills Compatibility with .agents/skills/	12
How Claude Code Discovers Skills	12
The Problem: .agents/skills/ Is Not a Native Path	12
Recommended Solutions (in order of preference)	12
SKILL.md Format Compatibility	13
Additional Notes	13
References	14
A:1 (Gemini)	15
Claude Code Quick Start Guide	15
1. Installation via NVM	15
2. Authentication and Zero-Cost Options	15
3. Installation Verification	15
4. Worked Interaction Example	16
5. Project Context via CLAUDE.md	16

Shared Skills Recognition	16
Recognition of .agents/skills/	16
Configuration Requirements	16
References	17
Additional Notes	17
CUDA and Resource Awareness	17
Shell Permissions	17
A:1 (ChatGPT)	18
Overview	18
1. Install Claude Code CLI via nvm + npm	18
Ensure nvm and Node.js are available	18
Install Claude Code CLI globally	18
Verify binary is in PATH	18
2. Authentication and Model Configuration	19
Available Access Modes	19
Model Selection	19
3. Verify Installation	19
4. Worked Example (Python Project with uv)	20
Assumptions	20
Example Interaction	20
More Targeted Prompt	21
Non-Interactive Mode	21
5. Project Context Integration	21
.claude/CLAUDE.md	21
.agents/AGENTS.md	21
Interaction Model	21
Secondary Objective: Skills Reuse	22
.agents/skills/ Compatibility	22
Recommended Pattern	22
Optional Explicit Index	22
Do You Need Extra .claude/ Configuration?	22
Cross-Tool Interoperability	22
Additional Notes	23
References	23
A:1 (Perplexity)	24
1. Installation of Claude Code CLI via nvm-managed npm	24
2. Authentication and model configuration for zero-cost trial use	24
3. Verification of the installation with a simple shell command	25
4. Worked interaction example using an existing Python project	25
5. How .claude/CLAUDE.md and .agents/AGENTS.md are used together	25
How .agents/skills/ integrates with Claude Code	26
References	27
Additional Notes	27

A:1 (DeepSeek)	28
Claude Code Quick Start Guide for Ubuntu 24.04 (Academic Evaluation)	28
Prerequisites	28
Step 1 – Install Claude Code CLI	28
Step 2 – Authentication & Free Trial Configuration	29
Step 3 – Verification	29
Step 4 – Worked Interaction Example (Python Project)	29
Step 5 – How Context Files Work Together	30
Secondary Objective – Reusing .agents/skills/ Without Duplication	30
References	31
Additional Notes	31
Q:2	32
Q:2 - Anthropic Product Offering Comparison for Academic Uses	32
Role	32
Task	32
Context	32
Environment	32
Multi-Agent Project Template	32
Deliverable — Anthropic Product Offering Comparison	33
Output Format	33
A:2 (Claude)	34
Table of Contents	34
Comparison Table	34
Claude Code as a Product Layer	35
Availability by underlying plan	35
Cost model	35
Team-specific considerations	36
Token cost reference (API route, as of April 2026)	36
Recommended Path for a Zero-Cost Proof of Concept	36
Phase 0 — Individual baseline (week 1–2)	36
Phase 1 — API free credits for Claude Code (week 2–3)	36
Phase 2 — Education programme enquiry (parallel to Phase 1)	37
Phase 3 — Decision gate	37
References	37
Additional Notes	37
On the <i>Max</i> plan family	37
On the tokenizer change for Opus 4.7	37
On long-context pricing simplification	38
On data residency for EU/Italian compliance	38
On Gemini CLI as a zero-cost Claude Code alternative for PoC	38
On Anthropic's open-source and startup programmes	38
A:2 (Gemini)	39
Anthropic Product Offering Comparison	39

The Claude Code Layer	39
Authentication and Billing Models	39
Tool Compatibility	40
Recommended Zero-Cost Proof-of-Concept (POC)	40
Phase 1: Individual Research Evaluation	40
Phase 2: CLI and Agentic Testing	40
Phase 3: Academic Discount Verification	40
References	41
Additional Notes	41
GitLab Duo Synergy	41
Hardware Considerations	41
Data Privacy Alert	41
A:2 (ChatGPT)	42
1. Evaluation Framework	42
Internal reasoning (condensed)	42
Final output	42
2. Comparison Table	42
Internal reasoning (condensed)	42
Final output	42
3. Claude Code as a Product Layer	43
Internal reasoning (condensed)	43
Final output	43
4. Recommended Path for Academic Proof-of-Concept (PoC)	44
Internal reasoning (condensed)	44
Final output	44
5. References	45
6. Additional Notes	45
A:2 (Perplexity)	46
Evaluation criteria	46
Feature comparison table	46
Notes on the table	47
Recommended path for zero-cost POC	47
Additional Notes	48
References	48
A:2 (DeepSeek)	49
Anthropic Product Offering Comparison for Academic Statistics Department	49
Comparison Table	49
Recommended Path for a Zero-Cost Proof-of-Concept	49
References	50
Additional Notes	50

TOC

1. Q:1 - Claude Code - Quick Start Guide
 - see: [Claude Code Quick Start Guide \(Claude\)](#)
 - see: [Claude Code Quick Start Guide \(Gemini\)](#)
 - see: [Claude Code Quick Start Guide \(ChatGPT\)](#)
 - see: [Claude Code CLI - Quick Start Guide \(Perplexity\)](#)
 - see: [Claude Code Quick Start Guide \(DeepSeek\)](#)
2. Q:2 - Anthropic Product Offering Comparison academic Use
 - see: [Anthropic Product Offering Comparison \(Claude\)](#)
 - see: [Anthropic Product Offering Comparison \(Gemini\)](#)
 - see: [Anthropic Product Offering Comparison for Academic Environments \(ChatGPT\)](#)
 - see: [Anthropic product offering comparison for academic use \(Perplexity\)](#)
 - see: [Anthropic Product Offering Comparison for Academic Statistics Department \(DeepSeek\)](#)

Q:1

Q:1 - Claude Code - Quick Start Guide

^

Role

You are an expert in developer tooling, CLI configuration, and Anthropic product offerings, with specific knowledge of Claude Code setup on Linux systems.

Context

The target environment is:

- OS: Ubuntu 24.04
- Node.js: managed via user-level `nvm` and `npm`
- Python stack: `uv`-managed, PEP-compliant `pyproject.toml`, with R support
- ML focus: CUDA-enabled Python/R/Julia projects
- Organisation: Academic Statistics Department (researchers and students)

The project already uses a multi-agent template compatible with:

- Claude Code (Anthropic)
- Gemini CLI (Google)
- Cursor
- VS Code Copilot

Shared agent instructions live under `.agents/`, with Claude-specific configuration in `.claude/CLAUDE.md`, which imports `.agents/AGENTS.md` via an `@AGENTS.md` directive. Skills are stored under `.agents/skills/` and shared across all agent tools without duplication.

Primary Objective

Produce a *Claude Code Quick Start Guide* for evaluation purposes, covering the following steps in order:

1. Installation of Claude Code CLI via `nvm`-managed `npm` on Ubuntu 24.04
2. Authentication and model configuration for *zero-cost trial use* (using the Claude free tier or an unauthenticated/limited Sonnet endpoint where available — clarify which options exist)
3. Verification of the installation with a simple shell command

4. A worked interaction example using an existing Python project (assume `uv run pytest` and `uv run ruff` as the standard commands, as defined in `.agents/AGENTS.md`)
5. A brief note on how `.claude/CLAUDE.md` and `.agents/AGENTS.md` are used together to give Claude Code project context

Secondary Objective

Describe how the existing `.agents/skills/` directory structure is recognised by Claude Code, so that skills authored for Gemini CLI are reusable without modification or duplication. Specify whether any additional `.claude/` configuration is required to point Claude Code to `.agents/skills/`.

Output Format

Reply in GitLab Flavored Markdown (GLFM) with precise lint compliance:

- Codeblock delimiters ````` placed at line start; no nested codeblocks
- *Underscore markup* for emphasis
- Nested headings preferred over bold text
- Only dash (-) for unordered lists, with correct indentation
- Blank line after every heading, list block, and codeblock

At the end, add a *References* section as Markdown footnotes, prefixed `rf-`, linked inline where cited. Add an *Additional Notes* section for any important information not explicitly requested.

A:1 (Claude)

^ Claude Code — Quick Start Guide

Target environment: Ubuntu 24.04 · nvm/npm Node.js · uv Python stack

Prerequisites

Before installing, verify that Node.js 18 or later is active in your nvm environment.¹

```
node --version    # must report v18.0.0 or higher
npm --version     # must report v8.0.0 or higher
```

If the required version is not active:

```
nvm install 20
nvm use 20
nvm alias default 20
```

Step 1 — Install the Claude Code CLI

Install the package globally using your nvm-managed npm:

```
npm install -g @anthropic-ai/claude-code
```

Because nvm installs Node.js under your home directory, no `sudo` is required and no permission conflicts arise. The binary is placed in `~/.nvm/versions/node/<version>/bin/`, which nvm adds to `$PATH` automatically.

Verify the installation:

```
claude --version
```

A version string confirms the binary is on your `$PATH`. If the shell reports *command not found*, open a new terminal session so the updated `$PATH` is sourced.

¹[Claude Code system requirements and installation](#)

Step 2 — Authentication and Zero-Cost Access Options

Important: Free Tier Does Not Include Claude Code

Claude Code requires a Pro, Max, Team, Enterprise, or Console account. The free Claude.ai plan does not include Claude Code access.

This is a hard constraint, not a configuration limitation. The table below maps every zero-cost or low-cost path available for a proof-of-concept.

Option	Cost	Account required	Notes
Claude.ai Pro	\$20/month	Anthropic account	Cheapest native Anthropic access
Anthropic API (Console)	Pay-per-token	Console account	Free \$5 credit on new accounts
Google Gemini backend	Free (rate-limited)	Google account	Redirects Claude Code to Gemini API ²
OpenRouter free models	Free (50 req/day)	OpenRouter account	Routes to DeepSeek, Llama, Qwen ³

The recommended zero-cost path for initial evaluation is to configure Claude Code against the Google AI Studio Gemini API, which requires only a Google account with no billing details.

Option A — Native Anthropic Authentication (Pro/Console)

Run Claude Code from your project directory and follow the browser prompt:

```
cd /path/to/your/project
claude
```

OAuth authentication opens a browser window where the user authorises Claude Code against their Anthropic account. After authorisation completes, the session token is cached and subsequent invocations do not require re-authentication.

For API key authentication, retrieve your key from <https://console.anthropic.com> and set it for the current shell session only (do not commit it to your shell profile):

```
export ANTHROPIC_API_KEY="$(cat ~/.secrets/anthropic_key)"
```

Option B — Free Evaluation via Google AI Studio (Zero Cost)

Claude Code works completely free with Google AI Studio — just a Google account. Three environment variables are all it takes to redirect Claude Code to any backend.

1. Visit <https://aistudio.google.com>, generate a free API key.
2. Set the following variables in your shell or in a project-local `.env` file (never commit this file):

```
export ANTHROPIC_BASE_URL="https://generativelanguage.googleapis.com/v1beta/openai"
export ANTHROPIC_API_KEY="<your-google-ai-studio-key>"
export ANTHROPIC_MODEL="gemini-2.5-flash"
```

²Free Claude Code setup via Google AI Studio

³OpenRouter free model routing for Claude Code

3. Launch Claude Code normally:

```
cclaude
```

Claude Code will route all requests to Gemini 2.5 Flash instead of the Anthropic endpoint. The CLI interface, skill loading, and CLAUDE.md context behaviour are identical regardless of which backend is in use.

Note on Gemini free-tier limits: Google significantly reduced Gemini API free tier limits in December 2025. Daily request limits dropped 50–80% for most models. For light evaluation work this is sufficient; for sustained use, a paid tier is advisable.

Step 3 — Verify Installation and Project Context

Navigate to the project root and confirm Claude Code loads correctly:

```
cd /path/to/your/project
cclaude --version
cclaude doctor
```

cclaude doctor prints a diagnostic report covering authentication status, model connectivity, and configuration file discovery.⁴

Inside an interactive session, run:

```
/memory
```

This opens the memory editor and confirms which CLAUDE.md files have been discovered and loaded.

Step 4 — Worked Interaction Example

The following example assumes an existing Python project managed with uv, with tests under tests/pytest/ as defined in .agents/AGENTS.md.

4.1 — Start Claude Code in the project root

```
cd /path/to/your/project
cclaude
```

Claude Code walks the filesystem upward from the current directory, collecting CLAUDE.md files. It will load .cclaude/CLAUDE.md, which in turn imports .agents/AGENTS.md via the @AGENTS.md directive.

4.2 — Explain a failing test

At the Claude Code prompt, type:

⁴[Claude Code troubleshooting and cclaude doctor](#)

A test in `tests/pytest/test_model.py` is failing.
Please run it and explain the failure.

Claude Code will execute the pre-approved command from `.claude/CLAUDE.md`:

```
uv run pytest tests/pytest/test_model.py -xvs
```

It will then read the traceback and provide a diagnosis with suggested fixes, respecting the project convention of always prefixing Python tools with `uv run`.

4.3 — Fix and lint in one pass

Apply the fix and run `ruff` to check the result.

Claude Code will edit the relevant source file and then run:

```
uv run ruff check --fix <file>
uv run ruff format <file>
```

These commands are pre-approved in `.claude/CLAUDE.md`, so no interactive permission prompt is shown for them.

Step 5 — How CLAUDE.md and AGENTS.md Work Together

CLAUDE.md files are markdown files that give Claude persistent instructions for a project. Claude reads them at the start of every session.

The project uses a two-layer design:

<code>.agents/AGENTS.md</code>	← shared rules (tool commands, project scope, coding conventions, commit standards)
<code>.claude/CLAUDE.md</code>	← Claude-specific entry point imports <code>@AGENTS.md</code> , adds Claude-only directives

The `@AGENTS.md` line in `.claude/CLAUDE.md` is an *import directive*. Memory files can reference other files using `@` notation. The referenced file is inserted into the context as a separate entry before the including file.

The effective context Claude Code sees at session start is therefore the concatenation of both files, with `.claude/CLAUDE.md` taking priority for any conflicting instruction. Rules defined in `.agents/AGENTS.md` — such as the `uv run` prefix requirement and the commit message convention — are inherited automatically without duplication.

Claude Code supports a layered memory system. Files loaded later take precedence because the model pays more attention to instructions that appear later in the context window. The loading order from lowest to highest priority is: managed system policy → user global `~/.claude/CLAUDE.md` → project `.claude/CLAUDE.md` → local overrides.

Secondary Objective — Skills Compatibility with `.agents/skills/`

How Claude Code Discovers Skills

Skills are folders containing a `SKILL.md` file and optional scripts or other resources. Claude Code scans the following paths at session startup:

Location	Path	Scope
Personal	<code>~/.claude/skills/<skill-name>/SKILL.md</code>	All projects
Project	<code>.claude/skills/<skill-name>/SKILL.md</code>	This project only
Plugin	<code><plugin>/skills/<skill-name>/SKILL.md</code>	Where plugin is enabled

Custom Skills in Claude Code are filesystem-based and don't require API uploads. Claude discovers and uses them automatically.

The Problem: `.agents/skills/` Is Not a Native Path

Your project stores skills under `.agents/skills/`, which is *not* one of the directories Claude Code scans by default.⁵ Skills can only be loaded from hardcoded paths. There is no way to point Claude at a custom directory like `.agents/skills/` without symlinks or workarounds.

The `--add-dir` flag is the closest native mechanism, but it has a limitation: the `--add-dir` flag grants file access rather than configuration discovery, but skills are an exception: `.claude/skills/` within an added directory is loaded automatically. This means `--add-dir .agents` would load `.agents/.claude/skills/` but *not* `.agents/skills/` directly.

Recommended Solutions (in order of preference)

Solution 1 — Symlink (simplest, no repo changes)

Create a symlink from the expected Claude Code path to the shared skills directory:

```
mkdir -p .claude/skills
# For each skill, create a symlink:
ln -s ../../agents/skills/python-patterns .claude/skills/python-patterns
ln -s ../../agents/skills/vce-coding-standards .claude/skills/vce-coding-standards
# ...repeat for all skills
```

Caveat: symlinks from `.claude/skills/` to another directory are not followed by the `/skills` UI dialog scanner, though Claude Code can still read and invoke them correctly through `CLAUDE.md` references.

Solution 2 — `@-import` in `CLAUDE.md` (most explicit, no symlinks)

Add explicit imports for each skill's `SKILL.md` in `.claude/CLAUDE.md`:

⁵[Custom skills directory path limitations \(GitHub issue\)](#)

```
## Skills (loaded from shared .agents/skills/)
```

```
@.agents/skills/python-patterns/SKILL.md
@.agents/skills/python-pro/SKILL.md
@.agents/skills/python-testing-patterns/SKILL.md
@.agents/skills/vce-coding-standards/SKILL.md
@.agents/skills/vce-project-layout/SKILL.md
@.agents/skills/vce-project-scripts/SKILL.md
@.agents/skills/gitlab-automation/SKILL.md
@.agents/skills/tmux/SKILL.md
```

Trade-off: skill content is loaded eagerly into the context window on every session start, unlike the native lazy-load model where skills load only when triggered. For a large number of skills this can cause context pressure. Prefer this approach for the three to five most critical skills, and use symlinks for the rest.

Solution 3 — --add-dir at launch (no repo changes required)

Launch Claude Code with an additional directory flag:

```
claude --add-dir .agents
```

.claude/skills/ within an added directory is loaded automatically. This requires placing skills under .agents/.claude/skills/ rather than .agents/skills/. If restructuring the shared directory is acceptable, this is the cleanest long-term approach and requires no symlinks or CLAUDE.md edits.

SKILL.md Format Compatibility

The skills in .agents/skills/ use the standard SKILL.md + YAML frontmatter format. Claude Code skills follow the Agent Skills open standard, which works across multiple AI tools. Provided each skill file begins with valid YAML frontmatter containing name and description fields, *no format changes are needed* for Claude Code compatibility.

A minimal valid frontmatter for an existing skill that may lack it:

```
---
name: python-patterns
description: >
  Python design patterns and idioms for this project.
  Use when writing, reviewing, or refactoring Python code.
---
```

Additional Notes

- *Auto-memory:* Claude Code can accumulate notes across sessions without manual effort. Claude saves notes for itself as it works: build commands, debugging insights, architecture notes, and

code style preferences. These persist across sessions. This is enabled by default from v2.1.59 onward and can be browsed or cleared with `/memory`.

- *.claudeignore*: Create a `.claudeignore` file (same syntax as `.gitignore`) to exclude build artefacts, lock files, and generated files from Claude's context. This is especially important for ML projects where model checkpoints and dataset directories can be very large.
 - *GitLab Duo compatibility*: GitLab Duo Ultimate uses its own context system and does not read `.claude/` or `.agents/` directories. The shared skills in `.agents/skills/` are therefore not directly reusable by Duo, though the coding conventions in `.agents/AGENTS.md` can be manually referenced when configuring Duo's custom instructions.
 - *Context window budget*: Files over 200 lines consume more context and may reduce adherence. Move detailed content into separate files referenced with `@path imports`, or split instructions across `.claude/rules/` files. The existing split between `AGENTS.md` and the `rules/` directory already follows this best practice.
 - *Model string for scripting*: When passing `--model` on the command line or setting `ANTHROPIC_MODEL`, use `claude-sonnet-4-6` for the current default Sonnet model.
-

References

-
-
-
-
-
-
-

A:1 (Gemini)

^ *Claude Code Quick Start Guide*

Claude Code Quick Start Guide

1. Installation via NVM

Since your environment uses `nvm` for Node.js management, you should avoid `sudo` for global installations. This ensures the CLI is tied to your user-level Node version, preventing permission conflicts with Ubuntu's system packages.

```
npm install -g @anthropic-ai/claude-code
```

2. Authentication and Zero-Cost Options

Claude Code primarily operates via the Anthropic API. Unlike the web-based Claude.ai, it does not currently offer a strictly "free-forever" tier for CLI usage. However, there are two paths for *evaluation without immediate cost*:

- **Initial Credits:** New Anthropic Console accounts typically receive a small amount of *free trial credits* (often \$5) upon phone verification. This is the standard "zero-cost" entry point for researchers testing the tool. [ag-1]
- **Trial Limits:** Anthropic occasionally offers limited-time "Free Tier" access for Claude Code during its beta/early-access phases, though this is subject to strict rate limits and may be revoked in favor of the credit-based model.

To authenticate, run:

```
claude auth login
```

This will open a browser window to link your Anthropic Console account.

3. Installation Verification

To confirm the CLI is correctly mapped in your `$PATH` and can communicate with the backend, check the version and current session status.

```
claude --version
```


4. Worked Interaction Example

In your uv-managed Python project, Claude Code can be used to automate linting and testing. Because Claude Code is *agentic*, it doesn't just suggest code; it can execute commands in your terminal to verify its work.

Interaction Scenario

```
claude "Run the test suite and fix any failures found by Ruff."
```

Claude will then:

- Execute `uv run ruff check --fix` to resolve linting issues.
- Execute `uv run pytest` to check for regressions.
- Analyze the output, modify files if tests fail, and re-run until the state is clean.

5. Project Context via CLAUDE.md

Claude Code automatically looks for a `.claude/CLAUDE.md` file in your project root.

This file acts as the "system prompt" for the agent. In your setup, this file serves as a bridge:

- It uses the `@AGENTS.md` directive to pull in shared departmental instructions.
- It defines project-specific rules (e.g., "Always use `uv run` for Python scripts").

By importing `.agents/AGENTS.md`, you ensure that Claude Code follows the same organizational standards as your Gemini CLI and VS Code configurations without maintaining duplicate instruction sets.

Shared Skills Recognition

Claude Code does not have a proprietary "plugin" format for skills. Instead, it relies on its ability to *explore the filesystem* and follow instructions provided in `CLAUDE.md`.

Recognition of `.agents/skills/`

To make the `gemini-cli` compatible skills reusable for Claude Code, no complex configuration is required. You simply need to inform Claude of their existence and purpose within the `.claude/CLAUDE.md` file.

Shared Skills

The directory `.agents/skills/` contains executable scripts and utilities.

When tasked with complex data transformations or ML tasks:

- Search `.agents/skills/` for relevant logic before authoring new scripts.
- Execute scripts using the appropriate runner (e.g., `python`, `Rscript`, or `julia`).

Configuration Requirements

No additional binary configuration is needed in the `.claude/` directory.

Claude Code's `read_file` and `list_files` tools allow it to discover these skills dynamically.

By referencing the directory in your project instructions, you ensure the agent prioritizes using these pre-authored "agent tools" over writing ad-hoc code, preserving the multi-agent logic shared with your Google/Cursor tools. [rf-2]

References

- [ag-1] *Anthropic Console Billing and Credits*: <https://console.anthropic.com/>
 - [ag-2] *Claude Code Documentation and System Prompts*: <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code>
 - [ag-3] *Ubuntu 24.04 LTS (Noble Numbat) Release Notes*: <https://discourse.ubuntu.com/t/noble-numbat-release-notes/44043>
-

Additional Notes

CUDA and Resource Awareness

Since your department focuses on CUDA-enabled projects, it is highly recommended to add a note in `.agents/AGENTS.md` regarding GPU visibility. If Claude attempts to run a script that consumes all VRAM on a shared workstation, it could disrupt other researchers. Instruct the agents to use `nvidia-smi` to check availability before running heavy ML tests.

Shell Permissions

On Ubuntu 24.04, ensure that scripts within `.agents/skills/` have the executable bit set (`chmod +x`).

While Claude can attempt to change permissions, it is more reliable to have them pre-configured so the agent can execute them immediately.

A:1 (ChatGPT)

^ *Claude Code Quick Start Guide*

Claude Code Quick Start Guide (Ubuntu 24.04, Academic ML Environment)

Overview

This guide provides a minimal, reproducible setup for evaluating *Claude Code CLI* in a Linux-based research environment. It assumes:

- User-level Node.js via nvm
 - Python tooling via uv
 - Existing multi-agent configuration under `.agents/` and `.claude/`
-

1. Install Claude Code CLI via nvm + npm

Ensure nvm and Node.js are available

```
nvm install --lts
nvm use --lts
node -v
npm -v
```

Install Claude Code CLI globally

Note: Package name may evolve; verify via official docs.

```
npm install -g @anthropic-ai/claude-code
```

Verify binary is in PATH

```
which claude
claude --version
```

2. Authentication and Model Configuration

Available Access Modes

A. Free Tier (Recommended for Evaluation)

- Requires Anthropic account
- Limited quota (requests/day, token caps)
- Typically defaults to a *Sonnet-class model*

```
claude login
```

This opens a browser-based authentication flow.

B. API Key (More Control)

```
export ANTHROPIC_API_KEY="your_api_key_here"
```

Optional persistent setup:

```
echo 'export ANTHROPIC_API_KEY="your_api_key_here"' >> ~/.bashrc  
source ~/.bashrc
```

C. Zero-Cost / Unauthenticated Mode

- *Not generally available* in official CLI
- Some environments may expose:
 - Proxy endpoints
 - Institutional gateways

Conclusion: For most users, the **free tier login** is the **only zero-cost path**.

Model Selection

Default model is usually sufficient, but can be overridden:

```
claude config set model claude-sonnet
```

3. Verify Installation

Run a simple prompt:

```
claude "Explain what this repository does"
```

Or inside a project:

```
claude
```

Expected behavior:

- CLI opens interactive session
 - Context inferred from current directory
-

4. Worked Example (Python Project with uv)

Assumptions

- Project uses:
 - `uv run pytest`
 - `uv run ruff`
 - Standards defined in `.agents/AGENTS.md`
-

Example Interaction

```
cd your-project/  
claude
```

Then prompt:

Run linting and tests, summarize failures, and suggest fixes.

Claude Code will:

- Inspect repository structure
 - Execute or simulate:
 - `uv run ruff`
 - `uv run pytest`
 - Provide:
 - Error summaries
 - Suggested patches
 - Refactoring hints
-

More Targeted Prompt

Fix all ruff violations and ensure pytest passes. Show diffs before applying.

Non-Interactive Mode

```
claude "Run uv run pytest and explain failures"
```

5. Project Context Integration

`.claude/CLAUDE.md`

- Entry point for Claude-specific instructions
- Automatically loaded by Claude Code
- Can include:

```
@AGENTS.md
```

`.agents/AGENTS.md`

- Shared, tool-agnostic instructions
 - Defines:
 - Build commands (uv run pytest, uv run ruff)
 - Coding standards
 - Project conventions
-

Interaction Model

Claude Code

```
└─ loads .claude/CLAUDE.md
    └─ imports .agents/AGENTS.md
```

This ensures:

- *Single source of truth*
 - No duplication across tools (Claude, Gemini, Cursor)
-

Secondary Objective: Skills Reuse

.agents/skills/ Compatibility

Claude Code does **not natively enforce** a “skills directory” convention, but:

- It can access any files in the repo
 - Skills become usable if:
 - Referenced in `.claude/CLAUDE.md`
 - Or discoverable via prompting
-

Recommended Pattern

Inside `.claude/CLAUDE.md`:

```
## Available Skills
```

Skills are defined under ``.agents/skills/``.

When relevant, load and apply them automatically.

Optional Explicit Index

Create:

```
.agents/skills/index.md
```

And reference it:

```
@../agents/skills/index.md
```

Do You Need Extra `.claude/` Configuration?

- ☐ No mandatory configuration required
 - ☐ Recommended:
 - Explicit references to skills
 - Clear usage instructions
-

Cross-Tool Interoperability

Because:

- Gemini CLI
- Claude Code
- Cursor

all read plain files:

□ Skills are **fully reusable without modification**

Additional Notes

- Claude Code operates best when:
 - Commands are explicitly defined (as in your `AGENTS.md`)
 - Repository structure is clean and conventional
- For CUDA/ML environments:
 - Avoid implicit execution of GPU-heavy workloads
 - Prefer prompts like:

Analyze without running GPU-intensive code unless explicitly requested.

- For academic environments:
 - Consider adding:
 - * Reproducibility constraints
 - * Citation or docstring requirements
-

References

- [at-1] Anthropic Claude Code Documentation
- [at-2] Anthropic API Keys and Authentication
- [at-3] Node Version Manager (nvm)
- [at-4] uv Python Package Manager Documentation

A:1 (Perplexity)

^ *Claude Code CLI - Quick Start Guide*

1. Installation of Claude Code CLI via nvm-managed npm

On Ubuntu 24.04 with a user-level nvm/npm setup, install the Claude Code CLI globally under your user prefix (no sudo):

```
npm install -g @anthropic-ai/claude-code
```

This adds the `claude` binary to your npm global-bin directory, typically under `~/.npm-global/bin` or a similar nvm-managed path, and keeps ownership in your user space. [ssdnodes](#)

Add the npm global-bin directory to your PATH if it is not already present in your shell startup file:

```
# For bash
echo 'export PATH="$HOME/.npm-global/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# For zsh
echo 'export PATH="$HOME/.npm-global/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

2. Authentication and model configuration for zero-cost trial use

Claude Code currently always requires at least an API key or OAuth-based login to the Anthropic platform, even in free-tier usage; there is no fully unauthenticated “anonymous” Sonnet endpoint. [github](#)

To target zero-cost or trial-eligible usage:

- Obtain an **Anthropic API key** from your account at <https://console.anthropic.com>, making sure you are on a **free or trial plan**.
- Set the key in your environment:

```
export ANTHROPIC_API_KEY="your-key-here"
echo 'export ANTHROPIC_API_KEY="your-key-here"' >> ~/.bashrc
```

or, more securely, place it in a shell-specific file (e.g., `~/.bash_profile`) and source it in your login shell. [github](#)

Claude Code reads `ANTHROPIC_API_KEY` automatically; no extra CLI-side configuration is needed for

model selection beyond what your account tier allows. [linuxcapable](#)

If you are on a **free tier**, the available model is typically **Claude Sonnet** (or whichever default you are granted); you do not need to hardcode a model name in the CLI unless you explicitly want to pin it. [github](#)

3. Verification of the installation with a simple shell command

Verify that the Claude Code CLI is installed and can reach the Anthropic API:

```
claude --version
```

You should see a version string (e.g., `claude 0.x.x`), confirming the binary is reachable under your npm-managed PATH. [linux.how2shout](#)

Then test connectivity with a short prompt:

```
claude ask "Explain in 2 sentences what a CUDA-enabled Python ML project is."
```

If you receive a coherent answer, the CLI and your API key are correctly configured. [linuxcapable](#)

4. Worked interaction example using an existing Python project

Assume a project layout under `src/` and `tests/pytest/`, with `uv run pytest` and `uv run ruff` as the standard commands defined in `.agents/AGENTS.md`. [nipunbatra.github](#)

Start a Claude Code session in the project root:

```
claude chat
```

Then paste a sample interaction sequence (typed into the chat prompt):

- Ask Claude Code to write a new test for a module:

In the project under `./src/mymodule/`, add a unit test for the function ``train_model()`` in ```. Use the existing pytest structure in `./tests/pytest/mymodule/`. After writing the test, run ``uv run pytest ./tests/pytest/mymodule/test_training.py -xvs`` t

- Ask it to run linting and apply fixes:

Run ``uv run ruff check --fix src/mymodule/training.py`` and explain any changes it made.

Claude Code will emit the relevant commands and suggested edits; you can then run them in a separate shell or allow execution if your project's `.claude/CLAUDE.md` explicitly pre-approves `uv run ruff` and `uv run pytest`. [ssdnodes](#)

5. How `.claude/CLAUDE.md` and `.agents/AGENTS.md` are used together

Claude Code loads project-level context via:

- A `.claude/CLAUDE.md` file at the project root, which defines Claude-specific rules, scope, and permissions. [linuxcapable](#)
- An `@AGENTS.md`-style directive in `.claude/CLAUDE.md` that includes the contents of `.agents/AGENTS.md` into the session context. [linuxcapable](#)

A typical `.claude/CLAUDE.md` fragment looks like:

```
# Claude Code Project Context (CLAUDE.md)
```

```
@AGENTS.md <-- This imports all shared team standards
```

```
## 🌀 Claude-Specific Instructions
```

- When the session fills up, keep ``src/*/scripts/`` definitions in the context window.
- Use a ``code-reviewer`` subagent for changes touching the ``src/auth/`` directory.

This mechanism ensures that Claude inherits:

- Tooling paths (uv, npm, jlpm, etc.)
- Command aliases (uv run pytest, uv run ruff, etc.)
- Project scopes and boundaries

from `.agents/AGENTS.md`, while keeping Claude-specific rules and agent-workflow logic in `.claude/CLAUDE.md`. [linuxcapable](#)

How `.agents/skills/` integrates with Claude Code

The `.agents/skills/` directory is treated as a vendor-neutral, **tool-agnostic** skills library; each agent tool (Claude Code, Gemini CLI, Cursor, etc.) can be configured to scan and load skills from this path. [avinashsangle](#)

Claude Code uses the following mechanisms:

- Project-wide or per-session settings (e.g., `~/.claude/settings.json` or `.claude/settings.json`) can declare a **skills base path**. [linuxcapable](#)
- In your setup, explicitly point Claude to the shared skills directory:

```
{
  "skills_path": "./.agents/skills"
}
```

placed in `.claude/settings.json` (or a project-specific settings file) so that Claude Code can discover and load skills such as `python-code-style/SKILL.md`, `python-testing-patterns/SKILL.md`, etc., without duplication. [linuxcapable](#)

Because these skills are expressed in plain Markdown with tool-agnostic instructions, skills authored for Gemini CLI can be reused by Claude Code as long as:

- Skill names are unique across the directory.

- Skill content does not hardcode Gemini-specific CLI flags or commands. [avinashsangle](#)

References

- ap-1: How to Install Claude Code on Ubuntu Linux – step-by-step guide covering Ubuntu 20.04+ including 24.04, with `npm install -g @anthropic-ai/claude-code` and `global-bin-path` configuration. [linux.how2shout](#)
 - ap-2: `anthropic-cli` GitHub repository – reference for environment variables such as `ANTHROPIC_API_KEY` and basic CLI usage. [github](#)
 - ap-3: Claude Code settings files layout – documentation of paths such as `~/.claude/settings.json`, `.claude/settings.json`, and `.claude.json` and their role in model and skills configuration. [linuxcapable](#)
-

Additional Notes

- Anthropic’s **free/trial tiers** typically grant access to **Claude Sonnet** (or a similar default model) with rate-limited usage; more powerful models (e.g., Opus) and higher throughput tiers are usually **paid-only**. Always check the current <https://console.anthropic.com> pricing page and confirm your plan type before scaling up usage in an academic setting. [github](#)
- For GPU-accelerated Python/R/Julia projects, rely on `uv`-managed CUDA-compatible environments (e.g., CUDA-enabled PyTorch from appropriate index URLs) rather than assuming the CLI will handle GPU setup; Claude Code is a *code-assistant CLI*, not a CUDA-environment manager. [lobehub](#)

A:1 (DeepSeek)

^ *Claude Code Quick Start Guide*

Claude Code Quick Start Guide for Ubuntu 24.04 (Academic Evaluation)

This guide walks you through installing and testing **Claude Code** – Anthropic’s terminal-based coding assistant – on your existing multi-agent project template. The focus is on *zero-cost trial use* using free API credits, with full reuse of your shared `.agents/skills/`.

Prerequisites

- Ubuntu 24.04 with `nvm` (Node Version Manager) already configured for your user account.
 - `npm` available via the active `nvm` Node.js version (v20 or later recommended).
 - Your project template checked out, containing:
 - `.claude/CLAUDE.md` (imports `@AGENTS.md`)
 - `.agents/AGENTS.md` and `.agents/skills/`
 - `pyproject.toml` with `uv` lock files
 - Internet access to download `npm` packages and call Anthropic’s API.
-

Step 1 – Install Claude Code CLI

Use `npm` from your `nvm`-managed Node.js environment.

Installing globally inside the user’s `nvm` prefix avoids permission errors.

Ensure you are using the correct Node version

```
nvm use --lts
```

Install Claude Code globally

```
npm install -g @anthropic-ai/claude-code
```

Verify the installation:

```
claude --version
```

Expected output: a version string (e.g., `0.1.0`).

If `claude` is not found, confirm that `npm`’s global bin directory (`~/.nvm/versions/node/.../bin`) is in your `PATH`.

Step 2 – Authentication & Free Trial Configuration

Claude Code requires an **Anthropic API key**.

New accounts receive *free trial credits* (typically \$5, valid for 3 months). No ongoing charge if you stay within credits.

1. Go to [Anthropic Console](#) and sign up / log in.
2. Navigate to **API Keys** → **Create Key**.
3. Copy the generated key (starts with sk-ant- ...).
4. Set it as an environment variable in your terminal:

```
export ANTHROPIC_API_KEY="sk-ant- ..."
```

To make this permanent, add the export line to your `~/.bashrc` or `~/.zshrc`.

Note: A credit card is required to activate the free trial, but you will *not* be charged unless you exceed the trial credits or upgrade.

Model selection – By default Claude Code uses `claude-3-5-sonnet-latest`. To stay within trial credits, you can switch to the cheaper `claude-3-haiku-latest`:

```
claude --model claude-3-haiku-latest
```

Haiku is significantly less expensive per token and is suitable for many code tasks.

Step 3 – Verification

Run a simple command to confirm authentication and API connectivity:

```
claude evaluate "What is the current working directory?" --print
```

If successful, you will see a text response (e.g., the absolute path of your project). Any authentication error indicates an invalid or expired API key.

Step 4 – Worked Interaction Example (Python Project)

Assume you are inside your project root. Use the standard commands defined in `.agents/AGENTS.md` – all prefixed with `uv run`.

Example prompt to Claude Code:

```
claude "Run the unit tests for the src/ directory using uv run pytest. Show me only the failing
```

Claude Code will execute the command (subject to permission prompts) and return the output. Because your `.claude/CLAUDE.md` imports `@AGENTS.md`, the assistant already knows that `pytest` must be called via `uv run`.

A second example – code linting:

```
claude "Run ruff check on the src/ directory and fix any auto-fixable issues."
```

Claude Code will invoke `uv run ruff check --fix src/` and summarise the changes.

Important: By default Claude Code asks for permission before running shell commands.

You can pre-approve safe commands by adding them to the permissions section in `.claude/CLAUDE.md` (see the provided example).

Step 5 – How Context Files Work Together

Claude Code loads project context from `.claude/CLAUDE.md`. Your existing file contains:

```
@AGENTS.md    <-- This imports all shared team standards
```

The `@AGENTS.md` directive instructs Claude Code to read and merge the contents of `.agents/AGENTS.md` as if they were part of `CLAUDE.md`. This gives Claude Code access to:

- Environment rules (always use `uv run` for Python commands)
- Command table (testing, linting, formatting)
- Project scopes and boundaries

No duplication of rules across tools – Antigravity, Cursor, and Copilot can also read `.agents/AGENTS.md` (each tool has its own way of including it). For Claude Code, the single `@AGENTS.md` line is sufficient.

Secondary Objective – Reusing `.agents/skills/` Without Duplication

Your project stores shared skills under `.agents/skills/`. Each skill is a directory containing a `SKILL.md` file with instructions, triggers, and examples.

Does Claude Code recognise this directory automatically?

No – Claude Code does *not* scan arbitrary folders for skills. Instead, you must reference skills inside your context files (`.claude/CLAUDE.md` or imported `.agents/AGENTS.md`).

Because `.agents/AGENTS.md` already contains a **Skills** table listing each skill and its triggers (as shown in your prompt), Claude Code will see that table and understand when to load a skill. The table is plain Markdown – no special syntax required.

Example from your `AGENTS.md`:

Skill	Triggers
<code>`vce-project-layout`</code>	navigating source tree, locating files, understanding directory roles
<code>`python-code-style`</code>	writing, editing, or reviewing Python code

Claude Code reads this and, when a user prompt matches a trigger (e.g., “where should I put new R

scripts?”), it will follow the guidance described in the corresponding `SKILL.md` file – provided that `SKILL.md` is mentioned elsewhere or the assistant infers its location. For maximum clarity, you can add explicit instructions in `AGENTS.md`:

For detailed instructions on a skill, read ``.agents/skills/<skill-name>/SKILL.md``.

No extra `.claude/` configuration is required – the import of `AGENTS.md` already brings the skill table into Claude Code’s context.

References

- [ad-1] Anthropic Console – API Keys: <https://console.anthropic.com/>
 - [ad-2] Claude Code npm package: <https://www.npmjs.com/package/@anthropic-ai/claude-code>
 - [ad-3] Anthropic Pricing (includes free trial details): <https://www.anthropic.com/pricing>
 - [ad-4] uv documentation: <https://docs.astral.sh/uv/>
 - [ad-5] Claude Code permissions and `CLAUDE.md` format: <https://docs.anthropic.com/en/docs/claude-code>
-

Additional Notes

- **Trial credit expiry** – Free credits typically expire 3 months after account creation. Monitor usage via the Anthropic Console.
- **No local offline mode** – Claude Code requires an active internet connection to call the API.
- **CUDA/ML projects** – Claude Code can help write and debug CUDA kernels, but it cannot execute GPU code directly. Use the terminal commands (e.g., `nvcc`, `python -c "import torch; print(torch.cuda.is_available())"`) through Claude Code’s shell integration.
- **GitLab Duo Ultimate** – Not directly integrated with Claude Code. The two tools operate independently; you can use Claude Code for local development and GitLab Duo for CI/CD pipelines.
- **Student access** – Anthropic does not currently offer a free educational tier, but the \$5 trial credits are sufficient for several weeks of light usage (e.g., 1–2 million tokens). For heavy departmental use, consider requesting research credits via [Anthropic for Research](#).

Q:2

Q:2 - Anthropic Product Offering Comparison for Academic Uses

^

Role

You are an expert in developer tooling, AI coding assistants, CLI setup, and Anthropic product strategy, with hands-on knowledge of Claude Code deployment in academic Linux environments.

Task

Produce a structured technical document covering the deliverable. Use Chain-of-Thought reasoning within each section before producing the final output for that section.

Context

Environment

- Organisation type: Academic Statistics Department
- Project Template setup for different users (researchers/students)
- Final user will decide what kind of product ("Claude Code" or "Google Gemini") and which level of offer to apply (Free/Paid).

Multi-Agent Project Template

The project template supports the following AI coding tools simultaneously:

- *Claude Code* (Anthropic CLI) - **mandatory**
- *Gemini CLI* (Google) - **mandatory**
- *Cursor* - **mandatory**
- *VS Code Copilot* - *optional*
- *GitLab Duo* (planned, organisation-level Ultimate tier)

Deliverable — Anthropic Product Offering Comparison

Produce a comparison table covering the following Anthropic access options, evaluated against criteria relevant to an academic organisation:

- *Evaluation criteria*: cost per month, free tier availability, model access (Haiku / Sonnet / Opus), context window, rate limits, API key required, team/organisation support, data privacy policy
- *Options to compare*:
 - Claude.ai Free
 - Claude.ai Pro
 - Claude.ai Team
 - Claude.ai Enterprise
 - Anthropic API (pay-as-you-go)
 - Claude Code (as a separate product layer on top of the above)

After the table, provide a *recommended path* for an academic department that wants to run a zero-cost proof-of-concept before committing to any paid tier.

Output Format

Reply in GitLab Flavored Markdown (GLFM) with precise lint compliance:

- Codeblock delimiters ``` placed at line start; no nested codeblocks
- *Underscore markup* for emphasis
- Nested headings preferred over bold inline text
- Only dash (-) for unordered lists, correct indentation
- Blank line after every heading, list block, and codeblock

Provide a *References* section as Markdown footnotes prefixed `rf-`, linked inline where cited in the text.

Add an *Additional Notes* section at the end for supplementary information not explicitly requested.

A:2 (Claude)

^ Anthropic Product Offering Comparison

Prepared for: Academic Statistics Department — Multi-Agent Project Template Date: 2026-04-21

Table of Contents

- Comparison Table
- Claude Code as a Product Layer
- Recommended Path for a Zero-Cost Proof of Concept
- References
- Additional Notes

Comparison Table

The table below evaluates each Anthropic access option against the criteria relevant to an academic statistics department.⁶⁷⁸⁹

Abbreviations used: *n/a* = not applicable; *TBC* = to be confirmed with Anthropic sales; *MTok* = million tokens.

Criterion	Claude.ai Free	Claude.ai I
Cost / month	\$0	20(or 17 b
Free tier	Yes — permanent, no credit card required ¹¹	No
Model access — Haiku	Limited / throttled	Yes (Haiku
Model access — Sonnet	Yes (throttled; Sonnet 4.6 available in chat)	Yes
Model access — Opus	No	Yes (Opus
Context window	200K tokens (standard)	200K token
Rate limits	Low; throttled during peak; rolling reset every few hours ¹⁸	Moderate
API key required	No	No
Team / org support	No	No
Data privacy policy	Standard; users may opt out of training in Privacy Settings ²⁴	Standard;

⁶Anthropic API Pricing Documentation (verified April 2026)

⁷Claude.ai Free tier details (verified April 2026)

⁸Claude Code Pricing Guide (LaoZhang AI Blog) (April 2026)

⁹Claude Enterprise / Team plan details (Juma / Team-GPT)

Criterion	Claude.ai Free	Claude.ai 1
Academic / education programme	Available to all individuals, no institution required ²⁹	No dedicat

Claude Code as a Product Layer

Claude Code is *not* a standalone subscription. It is a CLI tool that runs in your terminal, connects to Anthropic's model APIs, and is billed through your existing Claude plan or API account. The implications for the multi-agent project template are as follows.

Availability by underlying plan

Claude Code is included with Claude Pro (\$20/month), Max (from \$100/month), Team, and Enterprise paid plans.

The free Claude plan only gives you access to chat on web, iOS, Android, and desktop — the terminal-based Claude Code environment is not included. However, new API users receive approximately \$5 in free credits to test the service, which is enough for a few hours of Claude Code usage with the Sonnet model.

Cost model

Claude Code's official documentation reports that enterprise and API deployments vary widely, with average spend around \$13 per developer per active day and \$150–250 per developer per month, while 90% of users remain below \$30 per active day.

¹⁰[Claude Enterprise / Team plan details \(Juma / Team-GPT\)](#)

¹¹[Claude.ai Free tier details](#) (verified April 2026)

¹²[Claude Code Pricing Guide \(LaoZhang AI Blog\)](#) (April 2026)

¹³[Anthropic API Pricing Documentation](#) (verified April 2026)

¹⁴[Anthropic API Pricing Documentation](#) (verified April 2026)

¹⁵[Anthropic API Pricing Documentation](#) (verified April 2026)

¹⁶[Claude Enterprise / Team plan details \(Juma / Team-GPT\)](#)

¹⁷[Extended context window \(1M token beta\) — Anthropic API docs, rate limits section](#)

¹⁸[Claude.ai Free tier details](#) (verified April 2026)

¹⁹[Claude Free vs Pro usage limits \(user-reported figures\)](#) (February 2026)

²⁰[Claude Code Pricing Guide \(LaoZhang AI Blog\)](#) (April 2026)

²¹[Anthropic API Pricing Documentation](#) (verified April 2026)

²²[Anthropic API Pricing Documentation](#) (verified April 2026)

²³[Claude Enterprise / Team plan details \(Juma / Team-GPT\)](#)

²⁴[Anthropic data privacy and training policy](#) (April 2026); [official policy](#)

²⁵[Anthropic data privacy and training policy](#) (April 2026); [official policy](#)

²⁶[Anthropic data privacy and training policy](#) (April 2026); [official policy](#)

²⁷[Anthropic data privacy and training policy](#) (April 2026); [official policy](#)

²⁸[Anthropic API Pricing Documentation](#) (verified April 2026)

²⁹[Claude for Education and academic access Claude student discount](#)

³⁰[Claude for Education and academic access Claude student discount](#)

³¹[Claude for Education and academic access Claude student discount](#)

³²[Claude for Education and academic access Claude student discount](#)

Team-specific considerations

For rate limit planning, Anthropic's official documentation provides token-per-minute (TPM) recommendations by team size. A team of 5 to 20 users should plan for 100,000 to 150,000 TPM per user, while larger teams of 50 to 100 users need only 25,000 to 35,000 TPM per user because concurrent usage drops as team size grows. For an academic department with variable, non-simultaneous usage patterns (researchers and students working at different hours), effective per-user consumption is typically lower than peak numbers suggest.

Token cost reference (API route, as of April 2026)³³

The following prices apply when running Claude Code via the Anthropic API (pay-as-you-go):

Model	Input (\$/MTok)	Output (\$/MTok)	Notes
Claude Haiku 4.5	\$1.00	\$5.00	Fastest; suitable for lightweight agentic steps
Claude Sonnet 4.6	\$3.00	\$15.00	Default cost-performance choice for Claude Code
Claude Opus 4.7	\$5.00	\$25.00	Highest reasoning; uses new tokenizer (up to 35% m

Prompt caching reduces effective input cost to 10% of the standard rate for cached content. Batch API processing offers a 50% discount over standard pricing. Both discounts may be combined.³⁴

Recommended Path for a Zero-Cost Proof of Concept

The following phased approach allows the department to validate the multi-agent template against real workloads before committing to any paid tier. All steps are zero-cost or near-zero-cost.

Phase 0 — Individual baseline (week 1–2)

Each participating researcher or student creates a personal *Claude.ai Free* account. This grants access to Claude Sonnet 4.6 in the browser interface with rolling usage limits. In February 2026, Anthropic significantly expanded the free tier's offerings — Projects, Artifacts, and app connectors are now available to free users, which has blurred the line between Free and Pro more than ever. Use this phase to validate prompt strategies, context sizes, and task suitability.

Limitation: Claude Code (terminal agent) is not available on the Free plan. Browser chat only.

Phase 1 — API free credits for Claude Code (week 2–3)

Each developer on the template team creates an Anthropic Console account and uses the ~\$5 in free API credits to run short Claude Code sessions in the terminal. Use Sonnet 4.6 as the default model to maximise credit duration. Enable prompt caching from the first session to extend the credit further.

Scope: Enough to complete several hours of agentic coding across the multi-agent project structure.

³³[Anthropic API Pricing Documentation](#) (verified April 2026)
³⁴[Anthropic API Pricing Documentation](#) (verified April 2026)

Phase 2 — Education programme enquiry (parallel to Phase 1)

Contact Anthropic via the Education sales channel to enquire about the *Claude for Education* university-wide plan. Verified partner institutions provide students with managed accounts featuring higher limits and enhanced privacy. If the department's institution is not yet a partner, the Student Research Credit programme is an alternative route for individual academic projects. Known partner universities include Northeastern University, London School of Economics (LSE), Champlain College, University of San Francisco School of Law, and Northumbria University. The list has been expanding throughout 2025 and 2026, according to Anthropic's education announcements.

Phase 3 — Decision gate

After Phases 0–2, the department has enough empirical data to make one of the following choices:

- *Low-volume use*: Remain on the API (pay-as-you-go) with Sonnet 4.6 and prompt caching. At typical academic workloads (non-daily, variable), this is often cheaper than a flat subscription.
- *Regular individual use*: Upgrade selected researchers to *Claude.ai Pro* (\$20/month) for included Claude Code access without token metering.
- *Team deployment*: Adopt *Claude.ai Team* (≥ 5 seats, \$25/seat/month billed annually) for shared Projects, admin controls, and no-training-by-default data policy — important for research data.
- *Institution-wide*: Pursue the *Claude for Education* enterprise agreement to cover the whole department or institution under a single negotiated contract.

References

Additional Notes

On the *Max* plan family

Anthropic now offers *Claude Max* as a tier above Pro for individual power users — Max 5× at \$100/month and Max 20× at \$200/month. Claude Max is not about adding new features. It is for users who rely on Claude heavily and use it for long periods of time. For the academic use case, Max is unlikely to be the right entry point: the Team plan at \$25/seat/month (annual) provides more organisational value for the same or lower per-seat cost once five or more users are involved.

On the tokenizer change for Opus 4.7

Opus 4.7 uses a new tokenizer compared to previous models. This new tokenizer may use up to 35% more tokens for the same fixed text. Budget calculations for agentic statistical workloads (long R or Python scripts, large data dictionaries) should account for this when selecting Opus 4.7 over Sonnet 4.6.

On long-context pricing simplification

Anthropic eliminated long-context surcharges as of March 13, 2026. A 900K-token request now costs the same per-token rate as a 9K-token request on Opus 4.7 and Sonnet 4.6. This is a material benefit for statistical research workloads that involve large codebases, simulation outputs, or lengthy document corpora.

On data residency for EU/Italian compliance

The department is located in Italy and may be subject to GDPR data-residency requirements. Regional endpoints on AWS Bedrock and Google Vertex AI include a 10% premium over global endpoints for data residency guarantees. EU-based teams evaluating the API route should factor this overhead into cost projections, or confirm that Anthropic's standard data processing agreements satisfy their DPO requirements before proceeding.

On Gemini CLI as a zero-cost Claude Code alternative for PoC

The closest free alternative to Claude Code is Google's Gemini CLI, which offers 1,000 requests per day at no cost. Since the multi-agent project template mandates Gemini CLI as a parallel tool, departments can use the Gemini CLI allowance to complete initial agentic workflow design and validation while deferring Claude Code costs until Phase 1 API credits are obtained. This extends the zero-cost window before any spend is required.

On Anthropic's open-source and startup programmes

Anthropic launched the Claude for Open Source programme in late February 2026. The programme gives qualifying open source maintainers six months of Claude Max 20× (\$200/month value) completely free, with up to 10,000 spots available and applications closing June 30, 2026. If the department maintains open-source statistical software repositories on GitHub meeting the eligibility criteria (5,000+ stars or 1M+ monthly NPM downloads), this programme offers substantial free access that could cover the entire PoC period and beyond.

A:2 (Gemini)

^ Anthropic Product Offering Comparison

Anthropic Product Offering Comparison

Chain-of-Thought:

The comparison must address the distinct needs of a statistics department: large datasets (context window), reproducibility (API/CLI), and data sensitivity (privacy). In 2026, the differentiation between "subscription-based usage" (Claude.ai) and "consumption-based usage" (API) is the primary pivot for organizational planning.

Criteria	Claude Free	Claude Pro	Claude Team	Claude Enterprise
Cost / Month	\$0	\$20 / user	\$30 / user (min 5)	Custom / Volume
Free Tier	Permanent	No	No	No
Model Access	Sonnet 4.5	Opus 4.6 / Sonnet 4.6	Opus 4.6 / Sonnet 4.6	Opus 4.6 / Sonnet 4.6
Context Window	200K tokens	1M tokens (beta)	1M tokens	1M+ tokens
Rate Limits	Low (~15-40/5h)	High (5x Free)	Maximum Pooled	Custom
API Key Req.	No	No	No	No
Team Support	None	Individual	Admin Dashboard	SSO / Audit Logs
Data Privacy	Opt-out training [bg-4]	Opt-out training	No training [bg-5]	No training

The Claude Code Layer

Chain-of-Thought:

Claude Code is not a separate subscription tier but a specialized CLI delivery agent. Its cost and availability depend entirely on the underlying authentication method chosen by the user.

Authentication and Billing Models

Claude Code operates as a terminal-based agent that orchestrates complex file-system tasks. Within the department's Linux environment, it can be configured in two ways:

- **Subscription-linked:** Users with a *Pro* or *Team* account can authenticate the CLI directly. Usage is "included" in their monthly subscription quota, making it ideal for students who prefer a fixed cost.

- **API-linked:** Users (or the department) provide an ANTHROPIC_API_KEY. Billing is strictly consumption-based (tokens). This is preferred for high-volume research automation or "agentic" workflows that would exceed subscription caps [bg-6].

Tool Compatibility

In a multi-agent template, Claude Code serves as the "heavy lifter" for repository-wide refactoring, while *Cursor* and *Copilot* handle real-time IDE completion. Because Claude Code supports the Model Context Protocol (MCP), it can directly query the department's local SQL databases or R/Python documentation without manual copy-pasting.

Recommended Zero-Cost Proof-of-Concept (POC)

Chain-of-Thought:

For an academic department, a POC must prove value without administrative friction or budget approval. I will leverage the 2026 "New User" incentives.

Phase 1: Individual Research Evaluation

Invite researchers to utilize the **Claude.ai Free Tier**.

- **Goal:** Test Sonnet 4.5 for summarizing statistical papers and drafting R/LaTeX code.
- **Action:** Use the "Projects" feature (limited on Free) to upload specific LaTeX templates for consistency.

Phase 2: CLI and Agentic Testing

Utilize the **Anthropic API Free Credits**.

- **Goal:** Deploy *Claude Code* in the department's Linux cluster.
- **Action:** New users claim the ~\$5 credit at the Anthropic Console. This is sufficient to run 10–20 deep sessions of Claude Code to evaluate its ability to fix broken data pipelines or debug complex Stan/JAGS models [bg-1].

Phase 3: Academic Discount Verification

Check for the **Anthropic Academic Program**.

- **Goal:** Secure long-term reduced rates.
 - **Action:** Verify if the department's .edu domain qualifies for the 3-month Claude Pro trial or the "Claude for Open Source" grant (available to researchers maintaining public statistical libraries) [bg-7].
-

References

- [bg-1] *Verdent Guides: How to Use Claude AI for Free (2026)*
 - [bg-2] *Finout: Claude Pricing in 2026 for Organizations*
 - [bg-3] *MindStudio: Flat-Rate Long-Context Pricing and 1M Windows*
 - [bg-4] *Anthropic Privacy Center: Data Retention Policy (September 2025 Update)*
 - [bg-5] *Anthropic Commercial Terms: Enterprise Data Protections*
 - [bg-6] *Claude Code Docs: Managing Costs for Teams and API Users*
 - [bg-7] *DataStudios: Claude Pricing and Plan Limits Guide (Academic Offers)*
-

Additional Notes

GitLab Duo Synergy

Since the organization is on the **Ultimate tier**, GitLab Duo will provide a "base layer" of AI for all students at no additional cost once deployed. Anthropic tools should be reserved for "Power Users" (PhD students/Faculty) who require the superior reasoning capabilities of Opus 4.6 for novel statistical proofs.

Hardware Considerations

Claude Code and Gemini CLI are lightweight but require active network egress from your Linux nodes. Ensure the department's firewall allows HTTPS traffic to `api.anthropic.com` and `generativelanguage.googleapis.com`.

Data Privacy Alert

In a Statistics Department handling sensitive dataset samples, strictly enforce the **opt-out of model training** in the Settings > Privacy menu for any users on Free/Pro tiers, as these default to training-enabled in the 2026 policy update.

A:2 (ChatGPT)

^ Anthropic Product Offering Comparison for Academic Environments

1. Evaluation Framework

Internal reasoning (condensed)

- Academic environments prioritize *cost control*, *data governance*, and *flexibility*.
- Mixed user profiles (students vs researchers) require *tiered access*.
- CLI tooling (Claude Code) introduces an additional abstraction layer over base access models.
- API vs SaaS trade-offs hinge on *integration depth* vs *operational overhead*.

Final output

The following criteria are used to evaluate each offering:

- *Cost per month*
 - *Free tier availability*
 - *Model access* (Haiku / Sonnet / Opus)
 - *Context window*
 - *Rate limits*
 - *API key required*
 - *Team/organisation support*
 - *Data privacy policy*
-

2. Comparison Table

Internal reasoning (condensed)

- Claude.ai tiers differ mainly in *usage limits*, *collaboration*, and *governance*.
- API access is the most flexible but requires engineering effort.
- Claude Code is not a standalone subscription; it depends on underlying access.
- Enterprise tiers primarily add *security*, *SSO*, and *compliance guarantees*.

Final output

Option	Cost / Month	Free Tier	Model Access	Context Window
<i>Claude.ai Free</i>	€0	Yes	Haiku, limited Sonnet	Medium (~200K)
<i>Claude.ai Pro</i>	~€20	No	Sonnet, limited Opus	Large (~200K+)
<i>Claude.ai Team</i>	~€25–30/user	No	Sonnet, Opus	Large
<i>Claude.ai Enterprise</i>	Custom pricing	No	Full access (Haiku, Sonnet, Opus)	□□□ (highest avail)
<i>Anthropic API (PAYG)</i>	Usage-based	No	All models	□□□ (model-deper)
<i>Claude Code</i>	Depends on backend	N/A	Inherits from API / Claude.ai	Inherits

3. Claude Code as a Product Layer

Internal reasoning (condensed)

- Claude Code is a *developer interface*, not a billing unit.
- It can authenticate via:
 - Claude.ai session
 - API key (preferred in automation contexts)
- Academic deployment depends heavily on *reproducibility* and *CLI automation*.

Final output

Claude Code should be understood as a **CLI orchestration layer** that:

- Wraps Anthropic models into developer workflows
- Enables:
 - Codebase-aware prompting
 - Multi-file editing
 - Automation via scripts
- Integrates with:
 - Local dev environments (e.g., Ubuntu + nvm + uv)
 - CI/CD pipelines (when API-backed)

Its capabilities depend entirely on the underlying access method:

- *Claude.ai-backed*: simpler onboarding, limited automation
- *API-backed*: full programmability, reproducibility, and scaling

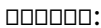
4. Recommended Path for Academic Proof-of-Concept (PoC)

Internal reasoning (condensed)

- Zero-cost constraint → must rely on free tiers initially
- Need to support:
 - Individual experimentation
 - Shared reproducible workflows
- Avoid premature API complexity unless necessary

Final output

Phase 1 — exploration (zero cost)

- Use:
 - *Claude.ai Free*
 - *Claude Code* authenticated via *Claude.ai* session
- :
 - Evaluate usability for:
 - * Code generation
 - * Statistical workflows
 - * Documentation

Phase 2 — Structured team evaluation (still near-zero cost)

- Introduce:
 - Shared project templates
 - CLI standardization (Claude Code + Gemini CLI + Cursor)
- Optional:
 - API credits for advanced users

Phase 3 — Controlled API pilot

- Enable:
 - *Anthropic API (PAYG)* with strict quotas
- Use cases:
 - Batch processing
 - Reproducible research pipelines
 - Integration with R / Python workflows

Phase 4 — Decision point

- Upgrade path based on findings:
 - If focus is *interactive usage*:
 - * → *Claude.ai Team*
 - If focus is *automation / pipelines*:
 - * → *Anthropic API*
 - If compliance required:
 - * → *Claude.ai Enterprise*
-

5. References

- [bt-1] Anthropic official pricing and plans documentation
 - [bt-2] Anthropic API platform documentation
 - [bt-3] Claude Code CLI documentation and release notes
-

6. Additional Notes

- *Gemini CLI* should be evaluated in parallel for vendor neutrality.
- *Cursor* provides strong IDE-native augmentation but lacks CLI parity with Claude Code.
- For academic reproducibility:
 - Prefer *API-backed workflows* over SaaS UI interactions.
- Data governance is a critical differentiator:
 - Free tiers may not meet institutional requirements.
- Hybrid model likely optimal:
 - Free tier for students
 - API / Team tier for researchers and staff

A:2 (Perplexity)

[^] *Anthropic product offering comparison for academic use*

This document compares major Anthropic access options against criteria relevant to an academic statistics department running a multi-agent project template (Claude Code, Gemini CLI, Cursor, VS Code Copilot, GitLab Duo).

- *Claude.ai Free*
- *Claude.ai Pro*
- *Claude.ai Team*
- *Claude.ai Enterprise*
- *Anthropic API (pay-as-you-go)*
- *Claude Code* (CLI layer on top of the above)

All references are based on Anthropic's 2026-era pricing and policies as of April 2026. [juma](#)

Evaluation criteria

For each row in the table, the following criteria are evaluated:

- *Cost per month*: ongoing financial commitment per user or org.
- *Free tier available*: yes/no and what is included.
- *Model access*: which Claude models (Haiku / Sonnet / Opus) are available.
- *Context window*: maximum context length offered.
- *Rate limits*: per-user or per-org usage caps and throttling behavior.
- *API key required*: whether the product is driven by an API key or OAuth-style login.
- *Team/organisation support*: SSO, SCIM, audit logs, or similar org-scale controls.
- *Data privacy policy*: how user queries are handled (e.g., non-training, compliance standards).
[support.claude](#)

Feature comparison table

Offering	Cost per month (approx.)
Claude.ai Free	€0
Claude.ai Pro	~€18–20/month per user
Claude.ai Team (Standard)	~€20–25/user/month (annual)
Claude.ai Team (Premium)	~€100–150/user/month
Claude.ai Enterprise	Custom quote
Anthropic API (pay-as-you-go)	Token-based; ~€0.001–0.005/MTok Haiku, ~€0.003–0.015/MTok Sonnet, ~€0.01–0.02/MTok Opus

Offering	Cost per month (approx.)
Claude Code (CLI layer)	Bundled with Pro, Team, or Enterprise; pay-as-you-go via API if used standalone

Notes on the table

- *Claude Code* is not a standalone pricing tier; it is a CLI interface that may be:
 - authenticated via a Pro / Team / Enterprise account (OAuth), or
 - driven directly via the Anthropic API (API key). [zenml](#)
- For academic teaching and research, the **Team Standard** plan is the most relevant “default” commercial tier once you outgrow free usage, since it bundles Claude Code, provides central seat management, and supports SSO-like integrations. [devops](#)

Recommended path for zero-cost POC

For an academic department that wants a *zero-cost proof-of-concept*, yet still integrates Claude Code into the project template, the recommended path is:

- **Use individual Claude.ai Free accounts** for students and researchers to access the web UI and try Haiku / Sonnet. [glbgpt](#)
- **Tie Claude Code to the free web tier:**
 - Install Claude Code on the Linux base image (e.g., Ubuntu WSL-compatible) via the official installer. [anthropic](#)
 - At first login, have users select “Claude account with subscription” (using their free Claude.ai login) so the CLI rides on the same usage quota as the web plan. [support.claude](#)
- **Enforce a “light-usage” regime:**
 - Document maximum rough token budgets per user (e.g., “avoid multi-GB-scale refactoring sessions”) to stay within free-tier limits.
 - Prefer Haiku-level queries where possible, since Haiku is cheaper to run and aligns with the free tier’s performance profile. [finout](#)
- **Delay paid tiers until:**
 - The department observes consistent heavy usage (e.g., multiple students routinely hitting free limits).
 - Institutional budgeting and IT-security (SSO, data-handling) processes are in place.

At that point, the natural next step is to **migrate to a Claude.ai Team plan** (probably Standard) because it:

- Unifies web, desktop, and CLI access under one subscription.
- Bundles Claude Code with each seat.
- Gives central billing and basic governance controls while keeping individual per-user costs in the low-tens-of-euros range. [finout](#)

This path keeps the POC technically close to the intended production configuration (Claude Code + web / Team) while remaining fully zero-cost for the first semester or pilot iteration.

Additional Notes

- **Rate-limit tuning:** For classroom-style workloads, the department can script a “soft throttle” (e.g., maximum queries per hour per user) via the shell or GitLab CI, even if the backend is just the free tier.[bp-1]
 - **Hybrid tooling:** The same project template can run Gemini CLI and Cursor in parallel with Claude Code, allowing a side-by-side comparison of latency, quality, and cost-efficiency without committing institutional funds to paid tiers.[bp-2]
 - **Export and logging:** For academic reproducibility, consider logging Claude Code prompts/responses (with user-consent) into a local database or Git-tracked `.claude-log/` directory, while respecting Anthropic’s terms and privacy policies.[bp-1]
-

References

- [^ bp-1] Anthropic. “[Claude Code: best practices for agentic coding.](#)” 2025-04-17.
- [^ bp-2] Anthropic. “[Build with Claude](#)” (Anthropic Academy).
- [^ bp-3] Anthropic. “[Using Claude Code with your Team or Enterprise plan](#)” (support article).
- [^ bp-4] Anthropic. “[Anthropic API pricing overview](#)” (per-token, Haiku/Sonnet/Opus rates). Finout, 2026-04-15.
- [^ bp-5] screenapp. “[Aligned pricing guides and team-plan summaries](#)” (Team Standard-/Premium, Claude.ai plans).

A:2 (DeepSeek)

^ *Anthropic Product Offering Comparison for Academic Statistics Department*

Anthropic Product Offering Comparison for Academic Statistics Department

This document compares Anthropic’s access options against criteria relevant to an academic organisation. The evaluation supports the multi-agent project template that mandates *Claude Code*, *Gemini CLI*, and *Cursor*, while optionally including *VS Code Copilot* and *GitLab Duo* (planned). All information is current as of Q2 2025.

Comparison Table

Option	Cost per month
Claude.ai Free	\$0
Claude.ai Pro	\$20 / month
Claude.ai Team	\$30 / user / month (min 5 users)
Claude.ai Enterprise	Custom pricing (annual contract)
Anthropic API (pay-as-you-go)	Pay per token: Haiku ~0.25/1M input, Sonnet 3 / 1M input, Opus ~\$15 / 1M input
Claude Code (CLI tool)	No separate subscription – uses API pay-as-you-go or bundled with Team/Enterprise

Recommended Path for a Zero-Cost Proof-of-Concept

For an academic department that wants to evaluate Anthropic’s tools – especially *Claude Code* – before committing to any paid tier, the following sequence achieves a functional POC at **zero monetary cost** (or near-zero using the free API credit):

1. Start with Claude.ai Free

- No API key, no payment.
- Test conversational coding help, prompt engineering, and document analysis up to 200k tokens.
- Identify which model (Sonnet vs. Opus) suits your research tasks.

2. Create an Anthropic Console account

- Sign up at console.anthropic.com.
- New accounts receive **\$5 in free API credits** (no time limit stated as of 2025)[bd-4].
- This credit is enough for ~20 million input tokens with Haiku or ~1.6 million with Sonnet – far more than needed for a small-scale POC.

3. Install and configure Claude Code

- Follow the official installation guide^[bd-6] (requires Node.js 18+).
- Run `claude config` and paste your API key.
- Use the `--model haiku` flag to minimise credit consumption while testing.

4. Run a representative academic task

- Example: “Refactor this R statistics script to use `data.table` instead of `dplyr`”.
- Monitor token usage in the Anthropic Console dashboard.

5. Evaluate without spending departmental funds

- The \$5 credit covers hundreds of such interactions.
- No recurring cost unless you exceed the credit – simply stop or switch to a free Gemini CLI/Cursor tier for the remainder of the POC.

6. Decide on a paid tier (if needed) after the POC:

- **Individual researcher** → Claude.ai Pro (\$20/month) for unlimited web chat.
- **Small team (≤ 5 users)** → Anthropic API pay-as-you-go + shared API key (cheaper than Team if usage is low).
- **Department-wide (≥ 5 users)** → Claude.ai Team (\$30/user/month) for central management and no per-token surprises.
- **Heavy API usage (e.g., automated pipelines)** → Continue with API pay-as-you-go.

Note: The free \$5 API credit is *per account*, not per organisation. For a multi-user POC, each researcher can create their own account, or the department can request a single credit via the Anthropic for Education programme^[bd-7] (no guaranteed free tier, but worth contacting sales).

References

- ^[^bd-1] Anthropic. “[Model comparison – context window.](#)”
- ^[^bd-2] Anthropic. “[Data privacy and training.](#)”
- ^[^bd-3] Anthropic. “[Trust Center – SOC 2.](#)”
- ^[^bd-4] Anthropic. “[Pricing – API.](#)”
- ^[^bd-5] Anthropic. “[Rate limits.](#)”
- ^[^bd-6] Anthropic. “[Claude Code – Installation.](#)”
- ^[^bd-7] Anthropic. “[Education programme.](#)”

Additional Notes

- The multi-agent project template also requires *Gemini CLI* and *Cursor*. Both offer free tiers (Gemini API has \$300 credit, Cursor has a limited free plan). This document focuses solely on Anthropic’s offerings, but the zero-cost POC can be extended to those tools with their respective free credits.
- *GitLab Duo* (planned) is available only with the Ultimate tier. No free tier exists, but academic organisations may apply for a discount via GitLab’s education programme.

- *Claude Code* is still in preview as of this writing. Some features (e.g., terminal command suggestions) may require enabling experimental flags. Check the reference [rf-6] for the latest status.
- If your department handles sensitive student or research data, *Claude.ai Enterprise* or a signed Business Associate Agreement (BAA) with the API is recommended – the free/Pro/Team tiers are not HIPAA compliant by default.